

★ DESKHUB CAPSTONE — DAILY CHECKLIST

Days 28 → 32 — Daily Tasks

Tick off each item. Refer to PROJECT_GUIDE.md for the full spec.

5 days

POST-TRAINING

~3 hrs

PER DAY

Day 32

FINAL DAY

How to use this file: each morning, open the day's card. Read the goal. Work through the tasks one by one. Verify the "Done when" criteria before moving on. Skim the pitfalls — they're real bugs we've seen others hit. End-of-day, push your code.

Day 28

Project Day 1 of 5

Foundations

Foundations & Login

Get the dev loop running. Build the login flow end-to-end. Token persists across reloads.

TASKS

- ☐ Run `npm install` then `npm run dev`; confirm both `:3001` and `:8080` are up
- ☐ Verify the API works — hit `http://localhost:3001/tickets` in a browser, see JSON
- ☐ Create `public/index.html` — login form (email, password, submit, error span). Semantic HTML.
- ☐ Create `public/styles/main.css` — basic reset + clean form styling
- ☐ Implement `src/utils/storage.js` — `get`, `set`, `remove`, `clear` (with `deskhub:` prefix)
- ☐ Implement `src/api/client.js` — `request` + `get` / `post` / `patch` / `del` shorthands; throw on non-2xx
- ☐ Implement `src/api/auth.js` — `login`, `logout`, `getCurrentUser`, `isAuthenticated`
- ☐ Implement `src/modules/auth.js` — wire up form submit, redirect on success
- ☐ Wire `src/main.js` to dispatch by `document.body.dataset.page`
- ☐ Build a placeholder `public/dashboard.html` with just "You're in!" and a logout button
- ☐ Wire logout button — clear storage, redirect to login

✓ **Done when:** log in as `priya@deskhub.in / demo123` → land on dashboard. Reload → still logged in. Wrong password → inline error. Logout → back to login.

⚠ Watch out for:

- ES modules need `<script type="module">` in HTML — without it, `import` fails
- `fetch` doesn't throw on 4xx/5xx — your wrapper MUST check `response.ok`
- Don't store the password — only token + user object

- `localStorage` values are always strings — `JSON.parse` on get, `JSON.stringify` on set

→ Tomorrow: real tickets list page with fetch, loading, error states

Day 29

Project Day 2 of 5

Tickets List

Tickets List + API Client

A real, fetching tickets list with proper loading, error, and empty states.

TASKS

- ☐ Create `public/tickets.html` — semantic table or grid container, search/filter slots empty
- ☐ Add `data-page="tickets-list"` on `<body>` ; wire init in `main.js`
- ☐ Implement `src/api/tickets.js` — `listTickets`, `getTicket`, `createTicket`, `updateTicket`, `deleteTicket`, `listComments`, `addComment`
- ☐ Implement `src/utils/formatDate.js` — `formatDate`, `formatDateTime`, `formatRelative` (use `Intl`)
- ☐ Implement `src/modules/tickets.js` — `initTicketsList`, `refresh`, `renderTable`
- ☐ Add a one-time fetch of `/users` at boot (cache it) — needed to show assignee names not IDs
- ☐ Render columns: ID, Title, Customer, Priority, Status, Assignee, Created
- ☐ Add a loading spinner/text visible while fetching
- ☐ Add error state with a retry button (handle case where json-server is offline)
- ☐ Add empty state ("No tickets found")
- ☐ Auth-protect the page — redirect to login if no token
- ☐ Style the table — readable, alternating rows, hover state

✓ **Done when:** all 30 tickets render in a clean table. Dates look like "7 May 2026", not the raw ISO. Assignee column shows names. Stop json-server → reload page → see error + working retry button.

⚠ Watch out for:

- Don't paginate yet — that's tomorrow. Fetch all 30 for now.
- Render in ONE DOM operation (build `innerHTML` or use a `DocumentFragment`) — not 30 `appendChild` calls
- `assignedTo` is an ID. Map it via the cached users list — don't fetch `/users` 30 times
- Use `textContent` for ticket titles, not `innerHTML` — Day 27 callback (XSS)

→ Tomorrow: search, filter, sort, paginate

Day 30

Project Day 3 of 5

List UX

Search · Filter · Sort · Paginate

Make the list usable at scale. Every interaction triggers a clean refetch with the right query string.

TASKS

- ❑ Implement `src/utils/debounce.js` — same shape as Day 27
- ❑ Define a single `state` object in `tickets.js` holding all filter values + page
- ❑ Add search input — wire with `debounce(refresh, 300)`
- ❑ Add status dropdown — "All" + open / in-progress / resolved / closed
- ❑ Add priority dropdown — "All" + low / medium / high / urgent
- ❑ Add assignee dropdown — populate from cached users list
- ❑ Add sort dropdown — Newest / Priority / Status
- ❑ Build a `buildQueryString(state)` helper — skip empty values
- ❑ Add pagination UI — Prev / page numbers (1, 2, 3 ...) / Next; 10 per page
- ❑ Read `X-Total-Count` from response headers to compute total pages
- ❑ Reset `page` to 1 whenever any filter or search changes
- ❑ Disable Prev on page 1; disable Next on last page
- ❑ *Stretch*: reflect filters in URL via `history.replaceState`; read on init

✓ **Done when:** type "login" → only matching tickets show. status=open + priority=urgent → narrowed list. Sort by priority → urgent first. Pagination shows "Page 2 of 3" correctly. Rapid typing fires only ONE fetch after you stop.

⚠ Watch out for:

- Don't fetch on every keystroke — that's why we debounce
- `X-Total-Count` is on `response.headers`, NOT in the body — read it BEFORE `.json()`
- Empty filter values should NOT be sent (don't build `?status=`)
- Returning state to page 1 on filter-change is critical — otherwise you're on page 4 of 1
- If you do the URL stretch, also handle the back/forward buttons (`popstate` event)

→ Tomorrow: detail page + create form + validation

Day 31

Project Day 4 of 5

Detail + Create

Ticket Detail View + Create Form

Full CRUD with validated forms, modals, and toasts.

TASKS

- ❑ Create `public/ticket-detail.html` — ticket info shell + comments shell + action buttons
- ❑ Implement `src/modules/ticketDetail.js` — `initTicketDetail`
- ❑ Read `?id=N` via `new URLSearchParams(location.search)`
- ❑ Load ticket + comments + users in parallel with `Promise.all`
- ❑ Render all ticket fields with proper formatting

- ❑ Implement `src/modules/ui.js` — toast (auto-dismiss 3s), basic modal (Esc + click-outside to close), `confirmDialog`
- ❑ Status dropdown that PATCHes on change — show toast on success
- ❑ Priority dropdown that PATCHes on change
- ❑ Assignee dropdown that PATCHes on change
- ❑ Build "New Ticket" button on tickets list — opens modal
- ❑ Implement `src/modules/form.js` — validators (required, minLength, maxLength, email, oneOf) + `validateField` + `validateForm`
- ❑ Wire validation on the create form: blur shows inline error, submit re-validates all, submit disabled when invalid
- ❑ On successful POST: close modal → refresh list → success toast
- ❑ Delete button on detail page → `confirmDialog` → DELETE → redirect to list

✓ **Done when:** click any row → detail page → change status → toast + reload → status persists. Click "New Ticket" → bad input → red inline errors → fix → submit → ticket appears in list. Click delete → "Are you sure?" → yes → ticket gone.

⚠ Watch out for:

- Detail-page fetches must be `Promise.all` — sequential awaits add up to ugly latency
- Validate on blur (per-field) AND on submit (all fields) — but not on every keystroke (jumpy UX)
- Show only the FIRST error per field, not all of them — keep the UI calm
- After a successful create, *don't* just append the new row locally — refresh the list (server is the source of truth)
- Use `textContent` for everything user-typed (titles, descriptions, comments) — XSS

→ Tomorrow: comments, dashboard, polish, ship

Day 32

FINAL DAY · Project Day 5 of 5

Ship It

Comments · Polish · README · Ship

Polish to a portfolio-grade artifact. Update the README. Push. Tag. Submit.

TASKS

- ❑ Add comments thread on detail page — list existing (sorted by `createdAt` asc), textarea + button to add new
- ❑ On comment submit — POST → re-fetch comments → render → clear textarea
- ❑ Build `public/dashboard.html` — 4 stat cards: Total / Open / In-Progress / Resolved
- ❑ Stats cards use the count from `X-Total-Count` for each filtered query (4 small fetches in parallel)
- ❑ Add "Recent 5 tickets" section on dashboard with link to detail pages
- ❑ Polish `ui.js` — toast queue (multiple stack), simple modal animation, full-screen loader for slow ops
- ❑ Add empty states everywhere they apply (no comments yet, no recent tickets, etc.)
- ❑ Polish CSS — consistent spacing scale, decent typography, responsive at 768px+, hover/focus states

- ❑ Pick at least ONE stretch goal: URL-state, optimistic updates, dark mode, keyboard shortcuts, CSV export
- ❑ Update `README.md` — replace template content with: your setup steps, 2–3 screenshots, architecture decisions, known limitations, "what I'd add next"
- ❑ Run a full smoke test from a clean clone: `git clone` in a fresh folder → `npm install` → `npm run dev` → walk through every flow
- ❑ Final commit, push to GitHub, tag `v1.0.0` with release notes
- ❑ Write a short reflection paragraph: *"What was hardest, what would I do differently?"*
- ❑ Email/Slack the trainer: GitHub URL + 1 screenshot + reflection paragraph
- ❑ Be ready for the 15-minute walkthrough call — practice demoing every feature in 5 minutes

✓ **Done when:** you'd be happy to put this on your CV. The README is a real document, not the template. Full flow works on a fresh clone. You can demo the whole app in 5 minutes without notes.

⚠ **Watch out for:**

- Don't start a stretch goal until the core is rock-solid — half-finished features look worse than no extra features
- Many devs ship great code with empty READMEs and lose points — the README is part of the deliverable
- Test from a FRESH clone before submitting — your local `node_modules` can hide install bugs
- Commit history matters — don't squash everything into one "final" commit; meaningful messages tell the story
- Don't forget the screenshots in README — visual evidence the app actually runs

★ **Ship it. Then book your walkthrough call.**

After submission

You'll have a 15-minute walkthrough call with the trainer. Be ready to:

- Demo every feature live (5 min)
- Explain WHY for each architectural decision (5 min)
- Answer "if you had another week, what would you add?" (1 min)
- Take feedback gracefully and ask one good question back (4 min)

The walkthrough matters as much as the code. **Practice it.**

What this project actually proves

Skill	Day(s) reinforced
Variables, types, operators	1–3
Conditionals, loops	4–5

Skill	Day(s) reinforced
Functions, callbacks	6
Array methods	7
Objects, destructuring, spread	8
ES6+ syntax	9
DOM + events	10–11
try/catch around async	12, 19
Closures (debounce, factory)	14
<code>this</code> binding	15
Classes (optional, in ui.js)	17
Fetch + async/await	18–19
Event-loop awareness (loaders)	20
Microtasks (Promise.all detail page)	20–21
ES modules	12, 24
Pure functions, immutability	25
Patterns (factory, observer-ish)	26
Debounce, XSS-safety, perf	27

When a recruiter asks "what JavaScript do you actually know?" — point them at this repo.